This document is currently completely overruled by docs/Elixir-and-Julia.md until further notice.

The unified Julia/Elixir language against Scale

The proposal in `docs/Elixir-and-Julia.md` is a complete, general-purpose programming language that hosts every construct of Julia and Elixir behind distinct tokens. It is not an extension of Scale's current surface. It is the semantic core that Scale compiles into.

The unified language is the programmable intermediate representation and semantic core one level below Basic English. Basic English is a restricted, declarative front-end that compiles into it. A programmer may write the unified language directly to use its full expressiveness; Basic English may use only a subset. The unified language owns its types, dispatch, values, modules, macros, and runtime.

Scale has no independent implementation or semantics separate from the unified language. Its features are implemented in the unified language. The current Scale implementation is the starting implementation of the unified language: parts are reused, parts are extended, and parts are replaced. There is no separate "Scale runtime" beside the unified runtime.

This document records the collision between the proposal and the current implementation, recommends the implementation stack, and names the tests that must exist before the refactor starts. It changes no code.

What the proposal is

The proposal has four commitments that decide every later question:

1. **Two dispatch constructs.** `method` is Julia's typed, unordered, specificity-resolved, open dispatch. `clause` is Elixir's ordered, first-match, pattern-and-guard dispatch. They are never the same keyword.
2. **A reified type hierarchy.** The runtime carries `Any`, `Number`, `Real`, `Integer`, `Bool <: Integer` and `Bool <: Atom`, `Symbol` distinct from `Atom`, `Nothing`, `Missing`, parametric types, unions, `where` constraints, abstract types, and primitive types.
3. **A general runtime.** It provides processes and messages, tasks and typed channels, OTP behaviours as groups, records as tagged tuples, bit syntax, modules, and two macro systems.
4. **Total coverage.** Every source construct is hosted. The only rejected programs are structurally invalid mixes of the two dispatch grammars, modifiers on bodies, and unmarked macros.

The single test is: if a programmer cannot tell from the keyword or operator alone which source language's semantics apply, the construct is not clean enough.

What Scale is today

The current implementation is a small, verification-oriented language and image runtime. It is the starting implementation of the unified language, not a separate Scale runtime.

1. **Surface.** Scale Basic English: controlled sentences such as "Every player that is alive can jump." The surface is deliberately restricted and refuses unnamed constructs by name.
2. **Values and execution.** Tensors, an append-only image, a stack, and a cooperative actor scheduler. Execution state is never serialized; the image is the durable authority.
3. **Types.** `DomainExpression.FieldType` has four sorts: `BooleanType`, `IntegerType`, `NaturalType`, and `RealType`. The verification lattice adds finite or symbolic coordinates and single-field narrowing guards.

4. **Dispatch.** Image-resident generic functions and methods (D19/W10), type-first over the stack's sorts, with extension-subset specificity and named ambiguity. There is no `clause`, no `where`, no open/closed modifier, and no reified parametric type.
5. **Verification.** The same document is emitted as an F# predicate and as an SMT obligation. Z3 reports `Proved`, `Falsified`, `Unknown`, or `Unavailable`; `Unknown` and `Unavailable` are never passes.
6. **Build.** One F# project, NativeAOT, no runtime code generation, no reflection-based execution, and a release gate of `just check`.

The reuse and gap register

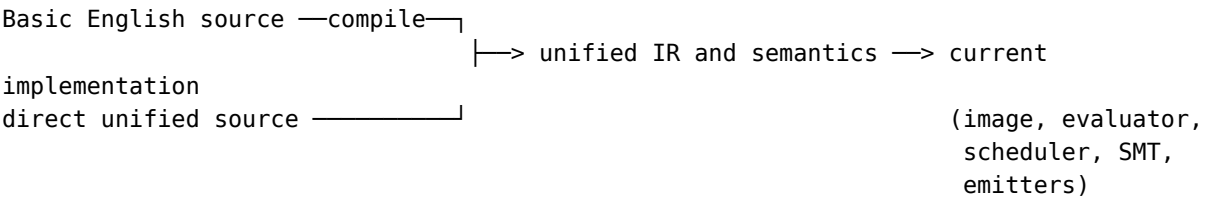
Area	Scale today	Proposal	Reuse and resolution
Surface	Scale Basic English: controlled, declarative specifications.	A general-purpose Julia/Elixir surface.	Basic English is a restricted front-end, not the whole language. It compiles into the unified language; direct programmers use the unified language for the full capacity.
Dispatch	`generic`/`method` as image-resident words; type-first over sorts.	`method` and `clause` as declaration keywords, with reified parametric types, `where`, open/closed heads, and guards.	The word `method` means two different things. The unified language owns both dispatch constructs and its tables. The current generic-function machinery is an implementation seed to assess, not a semantic definition.
Types	Four `FieldType` sorts; no `Any`, unions, parametric types, or subtyping.	Julia's type DAG, including `Bool <: Integer` and `Bool <: Atom`.	Names collide (`Real`, `Integer`, `Bool`) but meanings differ. The unified language owns its type lattice. `FieldType` remains a projection used by the verification feature; `Bool`'s two parents cannot be represented in it.
Values	Tensors, image entries, stack; append-only image.	General values, structs, mutable structs, records, closures, bit strings.	The current image and evaluator are the starting implementation, not a separate runtime. Assess which values fit

Area	Scale today	Proposal	Reuse and resolution
			the image and which need a general-heap extension; do not assume a second runtime.
Runtime	Cooperative local actors, channels, links, monitors, supervision.	Processes and messages, tasks and typed channels, OTP groups, modules, macros, bit syntax.	The current scheduler is the seed. It must be extended for tasks, OTP groups, modules, macros, and bit syntax; Scale has no separate runtime to preserve.
Errors	`EvaluationError` values with named reasons; blocking is a distinct outcome.	`raise`/`rescue`, `throw`/`catch`, and `exit` as three channels.	The unified language has its own outcome model. Current `EvaluationError`/ blocking values may be reused where they match; `exit` must not lower to `raise`.
Modules and macros	A flat vocabulary and image-resident words.	Two module systems and two macro systems.	Scale's vocabulary is not a module system. Modules and macros are unified language features; macro expansion must be compile-time for AOT.
Verification	SMT over declarative field predicates; `Proved`/`Falsified`/ `Unknown`/ `Unavailable`.	Arbitrary dynamic programs with macros, mutable state, and multiple dispatch.	Verification is a unified-language feature, implemented initially with the current lattice/SMT/ emitter code. Arbitrary programs are still not proved; the declarative subset is the contract surface.
Build and AOT	One F# project; no runtime code generation.	A compiler and runtime with reified types, dispatch tables, and macro expansion.	One implementation, not two. Table dispatch and compile-time macros are AOT-compatible; any `eval` or runtime code path is a separate JIT-only backend, outside the AOT entry point.

Area	Scale today	Proposal	Reuse and resolution
Plan	W0–W16, D1–D93, 1118 example checks, 78 granular tests.	New front-ends, IR, runtime, and a much larger test corpus.	Keep current tests as characterization. Add unified conformance and Basic English compilation tests separately; move checks only when a feature moves into the unified core.
Philosophy	Few concepts, explicit refusals, append-only image, failures as values.	Total coverage, no construct rejected, two constructs for differing semantics.	Basic English stays minimal; the unified language is complete. Because Basic English compiles into one unified core, both surfaces share one semantics and one dispatch model.
Transpiler	An F# emitter and an SMT emitter, not a general transpiler.	Julia and Elixir front-ends into the unified language.	The delayed Go-to-Scale path is a third, different direction. It is not the Julia/Elixir transpiler and must not be conflated with it.

The recommended split

The durable design is one implementation stack, not a Scale runtime beside a unified runtime.



The unified language is the programmable IR and the semantic core. Basic English compiles into it. A direct programmer writes it. There is one set of semantics and one implementation to extend.

The current implementation is the starting implementation of the unified language. Reuse is expected where the current semantics already match; extension is expected where the unified language is broader; replacement is expected where a current design contradicts the unified semantics. The decision is made per component, not by declaring a second runtime.

Why one implementation

Two implementations would duplicate dispatch, values, errors, and execution. The two would drift, and a Basic English program would not exercise the same machinery as a direct unified program. One semantic core removes that class of disagreement by construction.

Basic English still keeps its value: it is a small, controlled, declarative front-end that is easy to review and to verify. The unified language is larger because direct programmers need the full capacity. The difference is in the surface, not in the implementation.

What reuse means

1. The image, evaluator, scheduler, actor/channel machinery, lattice, SMT encoding, and emitters are candidates for reuse.
2. A component is reused only when its current behavior is the behavior the unified language specifies. Otherwise it is extended or replaced.
3. The current test suite becomes the characterization suite for the reused behavior. New unified tests state the target behavior.
4. The image stays append-only where it remains the value authority. A general
5. heap or mutable identity is added only where the unified semantics require it.

What must not be assumed

1. Do not assume a separate "Scale runtime" exists to preserve. Scale has no
2. independent implementation or semantics.
3. Do not assume the current image is already a general heap.
4. Do not assume the current `method` is the unified `method`, or that the
5. current vocabulary is a module system.
6. Do not assume verification covers arbitrary unified programs. Verification is
7. a unified-language feature for the declarative contract subset.

Invariants that must survive

1. Basic English remains a restricted, declarative surface. It compiles into the unified language; it is not executed by a separate runtime.
2. The unified language is the single programmable IR and semantic core. A direct unified program and a compiled Basic English program share one semantics and one dispatch model.
3. Scale has no independent implementation or semantics. The current implementation is the starting implementation of the unified language.
4. Reuse is per component and conditional: a component is reused only when its
5. current behavior matches the unified specification; otherwise it is extended
6. or replaced.
7. The image remains append-only where it is the value authority. A general heap
8. or mutable identity is added only where the unified semantics require it.
9. The unified language owns its type hierarchy, `method`/`clause` dispatch,
10. values, modules, macros, and concurrency. The current image-resident `method`
11. is an implementation seed, not the semantic definition.
12. Verification is a unified-language feature. It proves the declarative
13. contract subset only; `Unknown` and `Unavailable` are not passes.
14. `FieldType` remains the verification projection; it is not the unified
15. language's type system.
16. The release gate and NativeAOT contract apply to the one implementation.
17. Table dispatch and compile-time macros are AOT-compatible; `eval` and runtime
18. code generation are a separate JIT-only backend.

Test seams for the refactor

The first work item is tests, not implementation. The tests should pin the current implementation and state the unified target before the refactor moves either.

1. **Current-behavior characterization.** Tests for the image, evaluator,
2. scheduler, actors/channels, lattice, SMT outcomes, and emitters. These pass
3. today and become the baseline for the reuse decision.
4. **Basic English compilation.** For every Basic English construct, a
5. golden test that it compiles to the expected unified IR and preserves the
6. verification contract. These pin the current surface while the compiler is
7. introduced.
8. **Direct-versus-compiled equivalence.** A direct unified program and the
9. compiled equivalent Basic English document produce the same observable
10. result. This is the test that keeps one semantics.
11. **Reuse and gap tests.** For each current component, a test that states
12. whether its behavior matches the unified specification. A mismatch is a named
13. extension or replacement, not an unstated second implementation.
14. **Type projection.** A table test for the verification projection:
15. `Bool` to `BooleanType`; `Integer` to `IntegerType` or `NaturalType` only
16. with a proof; `Float64`/`AbstractFloat` to `RealType`; `Rational`,
17. `Complex`, `BigFloat`, `Symbol`, `Atom`, `Nothing`, and `Missing` to
18. `Unavailable`. The test must assert `Unavailable`, not a fallback.
19. **Dispatch target.** A test that the unified `method`/`clause` semantics
20. are the target, and that the current image-resident `method` is either
21. extended to match or replaced. The test names which one happened.
22. **Value and runtime target.** A test that a unified value is represented
23. by the current image where it fits, and a named gap where it does not. Do not
24. assume a second runtime.
25. **AOT boundary.** A structural test that the AOT entry point does not
26. reference a JIT-only backend, and that macro expansion happens before the
27. runtime.
28. **Source-to-unified differential tests.** For each construct in the
29. proposal's worked example, a Julia or Elixir input, the expected unified
30. construct, and a semantic assertion. The first batch covers `method`,
31. `clause`, `struct`, `|>`, `do`, keyword arguments, `with`, `case`,
32. `try`/`rescue`/`catch`, and the numeric tower.
33. **Structural-error tests.** `clause` with a type annotation, `method`
34. with a pattern or guard, modifiers on bodies, and unmarked macros are
35. compile errors, not runtime errors.

Staged plan

1. **Phase 0 — record the corrected model.** This document and D93. No
2. code change.
3. **Phase 1 — tests.** Current-behavior characterization, Basic English
4. compilation goldens, direct-versus-compiled equivalence, reuse/gap tests,
5. type projection, dispatch target, value/runtime target, and the AOT
6. boundary.
7. **Phase 2 — unified core on the current implementation.** Type
8. hierarchy, `method`/`clause` dispatch, values, and the worked example. Each
9. current component is reused, extended, or replaced with a test that says
10. which.
11. **Phase 3 — compile Basic English.** Basic English compiles to unified
12. IR. Its declarative contracts use the verification feature; there is no

13. separate Scale runtime.
14. **Phase 4 — front-ends.** Elixir first, because the current actor and
15. BEAM analysis is closer to it. Julia second, because the numeric tower and
16. parametric dispatch are larger.
17. **Phase 5 — close runtime gaps.** Tasks, OTP groups, modules, macros,
18. bit syntax, and the AOT backend, each implemented in the unified core.
19. **Phase 6 — coverage.** Add constructs in source order, each with a
20. distinct token and a differential test, until both ecosystems are covered.

Open owner decisions

1. **Reuse scope.** Which current components are reused as-is, extended, or
2. replaced? The recommendation is a per-component decision, starting with the
3. image, evaluator, scheduler, lattice, SMT encoding, and emitters.
4. **Runtime host.** Is the unified runtime the current .NET/NativeAOT
5. implementation, or a separate host? The recommendation is to extend the
6. current implementation; if .NET, table dispatch and compile-time macros are
7. mandatory.
8. **Value representation.** Does the current image carry the unified values
9. directly, or does it need a general-heap extension? The recommendation is to
10. extend the image where it fits and add a general heap only for values it
11. cannot represent.
12. **OTP distribution.** Does "100% coverage" include BEAM distribution and
13. the code server, or only local OTP semantics? The proposal specifies local
14. groups and processes; distribution is not named.
15. **First import target.** Elixir or Julia? The recommendation is Elixir
16. first.
17. **Verification scope.** Declarative contracts only, or arbitrary
18. programs? The recommendation is declarative contracts only; arbitrary
19. program verification would break the trust boundary.

Relationship to existing plans

The native .NET port (`docs/native-dotnet-port.typ`) is the plan for the one implementation, now understood as the unified-language implementation: executable IR, then an SRM writer, then self-typing. It is not a separate Scale Spec runtime. The Go-transpiler note (`docs/memory-and-go-transpiler.typ`) is a third direction and stays delayed. The BEAM compatibility matrix (`docs/beam-compatibility.typ`) describes the current local actor subset; it is input to the unified runtime's Elixir side, not a claim that the current implementation already is BEAM.

The unified language is the centerpiece: the programmable IR and semantic core that Basic English compiles into and that direct programmers use for the full capacity. The current implementation is its starting implementation.